MPNet: Masked and Permuted Pre-training for Language Understanding

Kaitao Song[1*] **Xu Tan**[2*] **Tao Qin**[2] **Jianfeng Lu**[1] **Tie-Yan Liu**[2]
[1]Nanjing University of Science and Technology
[2]Microsoft Research Aisa
{kt.song,lujf}@njust.edu.cn
{xuta,taoqin,tyliu}@microsoft.com
https://github.com/microsoft/MPNet

arXiv:2004.09297v1 [cs.CL] 20 Apr 2020

Abstract

BERT adopts masked language modeling (MLM) for pre-training and is one of the most successful pre-training models. Since BERT neglects dependency among predicted tokens, XLNet introduces permuted language modeling (PLM) for pre-training to address this problem. We argue that XLNet does not leverage the full position information of a sentence and thus suffers from position discrepancy between pre-training and fine-tuning. In this paper, we propose MPNet, a novel pre-training method that inherits the advantages of BERT and XLNet and avoids their limitations. MPNet leverages the dependency among predicted tokens through permuted language modeling (vs. MLM in BERT), and takes auxiliary position information as input to make the model see a full sentence and thus reducing the position discrepancy (vs. PLM in XLNet). We pre-train MPNet on a large-scale dataset (over 160GB text corpora) and fine-tune on a variety of down-streaming tasks (GLUE, SQuAD, etc). Experimental results show that MPNet outperforms MLM and PLM by a large margin, and achieves better results on these tasks compared with previous state-of-the-art pre-trained methods (e.g., BERT, XLNet, RoBERTa) under the same model setting. We release the code and pre-trained model in GitHub[1].

1 Introduction

Pre-training models (Radford et al., 2018; Devlin et al., 2019; Radford et al., 2019b; Song et al., 2019; Yang et al., 2019; Dong et al., 2019; Liu et al., 2019a; Raffel et al., 2019a) have greatly boosted the accuracy of NLP tasks in the past years. One of the most successful models is BERT (Devlin et al., 2019), which mainly adopts masked language modeling (MLM) for pre-training[2]. MLM leverages bidirectional context of masked tokens efficiently, but ignores the dependency among the masked (and to be predicted) tokens (Yang et al., 2019).

To improve BERT, XLNet (Yang et al., 2019) introduces permuted language modeling (PLM) for pre-training to capture the dependency among the predicted tokens. However, PLM has its own limitation: Each token can only see its preceding tokens in a permuted sequence but does not know the position information of the full sentence (e.g., the position information of future tokens in the permuted sentence) during the autoregressive pre-training, which brings discrepancy between pre-training and fine-tuning. Note that the position information of all the tokens in a sentence is available to BERT while predicting a masked token.

In this paper, we find that MLM and PLM can be unified in one view, which splits the tokens in a sequence into non-predicted and predicted parts. Under this unified view, we propose a new pre-training method, masked and permuted language modeling (MPNet for short), which addresses the issues in both MLM and PLM while inherits their advantages: 1) It takes the dependency among the predicted tokens into consideration through permuted language modeling and thus avoids the issue of BERT; 2) It takes position information of all tokens as input to make the model see the position information of all the tokens in the sentence and

[*]Authors contribute equally to this work. Correspondence to Tao Qin: taoqin@microsoft.com

[1]https://github.com/microsoft/MPNet

[2]We do not consider next sentence prediction here since previous works (Yang et al., 2019; Liu et al., 2019a; Joshi et al., 2019) have achieved good results without next sentence prediction.

thus alleviates the position discrepancy of XLNet.

We pre-train MPNet on a large-scale text corpora (over 160GB data) following the practice in Yang et al. (2019); Liu et al. (2019a), and fine-tune on a variety of down-streaming benchmark tasks, including GLUE, SQuAD, RACE and IMDB. Experimental results show that MPNet outperforms MLM and PLM by a large margin, which demonstrates that 1) the effectiveness of modeling the dependency among the predicted tokens (MPNet vs. MLM), and 2) the importance of the position information of the full sentence (MPNet vs. PLM). Moreover, MPNet outperforms previous well-known models BERT, XLNet and RoBERTa by 4.6, 3.2 and 1.3 points respectively on GLUE tasks under the same model setting, indicating the great potential of MPNet for language understanding[3].

2 MPNet

2.1 Background

The key of pre-training methods (Radford et al., 2018; Devlin et al., 2019; Song et al., 2019; Yang et al., 2019; Clark et al., 2020) is the design of self-supervised tasks/objectives for model training to exploit large language corpora for language understanding and generation. For language understanding, masked language modeling (MLM) in BERT (Devlin et al., 2019) and permuted language modeling (PLM) in XLNet (Yang et al., 2019) are two representative objectives. In this section, we briefly review MLM and PLM, and discuss their pros and cons.

MLM in BERT BERT (Devlin et al., 2019) is one of the most successful pre-training models for natural language understanding. It adopts Transformer (Vaswani et al., 2017) as the feature extractor and introduces masked language model (MLM) and next sentence prediction as training objectives to learn bidirectional representations. Specifically, for a given sentence $x = (x_1, x_2, \cdots, x_n)$, MLM randomly masks 15% tokens and replace them with a special symbol $[M]$. Denote $\mathcal{K}$ as the set of masked positions, $x_{\mathcal{K}}$ as the set of masked tokens, and $x_{\setminus\mathcal{K}}$ as the sentence after masking. As shown in the example in the left side of Figure 1(a), $\mathcal{K} = \{2, 4\}$, $x_{\mathcal{K}} = \{x_2, x_4\}$ and $x_{\setminus\mathcal{K}} = (x_1, [M], x_3, [M], x_5)$. MLM pre-trains the model θ by maximizing the following objective

$$\log P(x_{\mathcal{K}}|x_{\setminus\mathcal{K}};\theta) \approx \sum_{k\in\mathcal{K}} \log P(x_k|x_{\setminus\mathcal{K}};\theta). \quad (1)$$

PLM in XLNet Permuted language model (PLM) is proposed in XLNet (Yang et al., 2019) to retain the benefits of autoregressive modeling and also allow models to capture bidirectional context. For a given sentence $x = (x_1, x_2, \cdots, x_n)$ with length of n, there are $n!$ possible permutations. Denote $\mathcal{Z}_n$ as the permutations of set $\{1, 2, \cdots, n\}$. For a permutation $z \in \mathcal{Z}_n$, denote z_t as the t-th element in z and $z_{<t}$ as the first $t-1$ elements in z. As shown in the example in the right side of Figure 1(b), $z = (1, 3, 5, 2, 4)$, and if $t = 4$, then $z_t = 2$, $x_{z_t} = x_2$ and $z_{<t} = \{1, 3, 5\}$. PLM pre-trains the model θ by maximizing the following objective

$$\log P(x;\theta) = \mathbb{E}_{z\in\mathcal{Z}_n} \sum_{t=c+1}^{n} \log P(x_{z_t}|x_{z_{<t}};\theta), \quad (2)$$

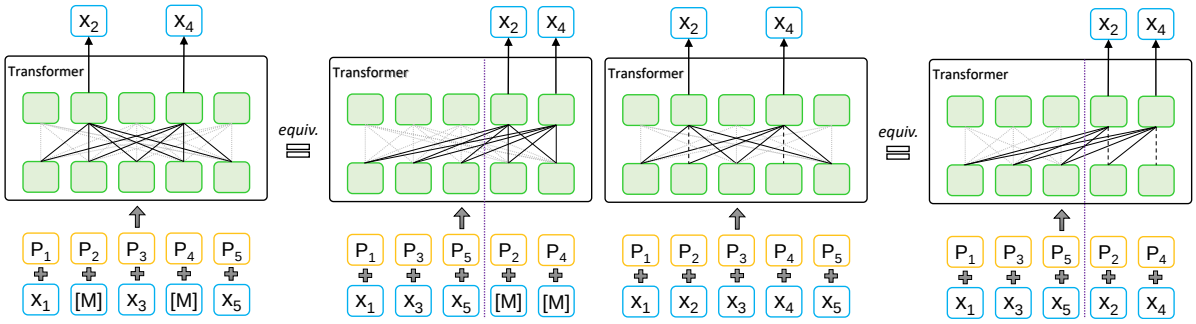
where c denotes the number of non-predicted tokens $x_{z_{<=c}}$. In practice, only a part of last tokens $x_{z_{>c}}$ (usually $c = 85\% * n$) are chosen to predict and the remaining tokens are used as condition in order to reduce the optimization difficulty (Yang et al., 2019).

Pros and Cons of MLM and PLM We compare MLM and PLM from two perspectives: the dependency in the predicted (output) tokens and the discrepancy between pre-training and fine-tuning in the input sentence.

- **Output Dependency**: As shown in Equation 1, MLM assumes the masked tokens are independent with each other and predicts them separately, which is not sufficient to model the complicated context dependency in natural language (Yang et al., 2019). In contrast, PLM factorizes the predicted tokens with the product rule in any permuted order, as shown in Equation 2, which avoids the independence assumption in MLM and can better model dependency among predicted tokens.

- **Input Discrepancy** Since in fine-tuning of downstream language understanding tasks, a model can see the full input sentence, to reduce

[3]We are still pre-training MPNet in larger model setting. We also welcome someone who has computation resources to pre-train larger MPNet models based on our released code to further advance the state of the art.



(a) MLM (b) PLM

Figure 1: A unified view of MLM and PLM, where x_i and p_i represent token and position embeddings. The left side in both MLM (a) and PLM (b) are in original order, while the right side in both MLM (a) and PLM (b) are in permuted order and are regarded as the unified view.

the discrepancy between pre-training and fine-tuning, the model should see as much information as possible of the full sentence during pre-training. In MLM, although some tokens are masked, their position information (i.e., the position embeddings) are available to the model to (partially) represent the information of full sentence (how many tokens in a sentence, i.e., the sentence length). However, each predicted token in PLM can only see its preceding tokens in a permuted sentence but does not know the position information of the full sentence during the autoregressive pre-training, which brings discrepancy between pre-training and fine-tuning.

As can be seen, PLM is better than MLM in terms of leveraging output dependency while worse in terms of pre-training and fine-tuning discrepancy. A natural question then arises: can we address the issues in both MLM and PLM while inherit their advantages?

2.2 A Unified View of MLM and PLM

To address the issues and inherit the advantages of MLM and PLM, in this section, we provide a unified view to understand MLM and PLM. Both BERT and XLNet take Transformer (Vaswani et al., 2017) as their backbone. Transformer takes tokens and their positions in a sentence as input, and is not sensitive to the absolute input order of those tokens, only if each token is associated with its correct position in the sentence.

This inspires us to propose a unified view for MLM and PLM, which rearranges and splits the tokens into non-predicted and predicted parts, as illustrated in Figure 1. For MLM in Figure 1(a), the input in the left side is equal to first permuting the sequence and then masking the tokens in rightmost (x_2 and x_4 are masked in the permuted sequence $(x_1, x_3, x_5, x_2, x_4)$ as shown in the right side). For PLM in Figure 1(b), the sequence $(x_1, x_2, x_3, x_4, x_5)$ is first permuted into $(x_1, x_3, x_5, x_2, x_4)$ and then the rightmost tokens x_2 and x_4 are chosen as the predicted tokens as shown in the right side, which equals to the left side. That is, in this unified view, the non-masked tokens are put in the left side while the masked and to be predicted tokens are in the right side of the permuted sequence for both MLM and PLM.

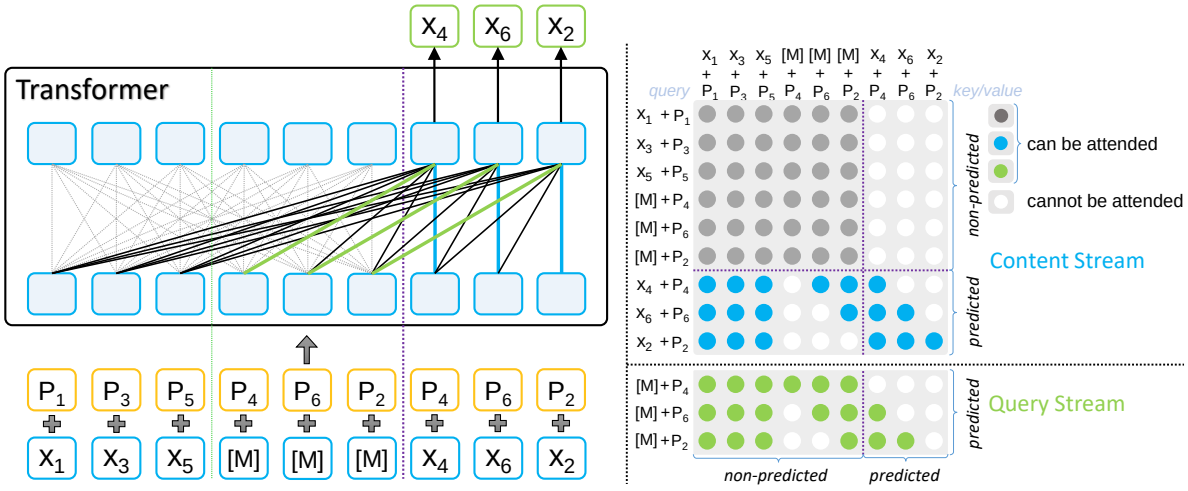
Under this unified view, we can rewrite the objective of MLM in Equation 1 as

$$\mathbb{E}_{z \in \mathcal{Z}_n} \sum_{t=c+1}^{n} \log P(x_{z_t} | x_{z_{<=c}}, M_{z_{>c}}; \theta), \quad (3)$$

where $M_{z_{>c}}$ denote the mask tokens [M] in position $z_{>c}$. As shown in the example in Figure 1(a), $n = 5$, $c = 3$, $x_{z_{<=c}} = (x_1, x_3, x_5)$, $x_{z_{>c}} = (x_2, x_4)$ and $M_{z_{>c}}$ are two mask tokens in position $z_4 = 2$ and $z_5 = 4$. We also put the objective of PLM from Equation 2 here

$$\mathbb{E}_{z \in \mathcal{Z}_n} \sum_{t=c+1}^{n} \log P(x_{z_t} | x_{z_{<t}}; \theta). \quad (4)$$

As can be seen from Equation 3 and 4, under this unified view, MLM and PLM share similar mathematical formulation but just with slight difference in the conditional part in $P(x_{z_t}|\cdot;\theta)$: MLM conditions on $x_{z_{<=c}}$ and $M_{z_{>c}}$, and PLM conditions on $x_{z_{<t}}$. In the next subsection, we describe how to modify the conditional part to address the issues and inherit the advantages of MLM and PLM.



(a) (b)

Figure 2: (a) The structure of MPNet. (b) The attention mask of MPNet. The light grey lines in (a) represent the bidirectional self-attention in the non-predicted part $(x_{z_{<=c}}, M_{z_{>c}}) = (x_1, x_5, x_3, [M], [M], [M])$, which correspond to the light grey attention mask in (b). The blue and green mask in (b) represent the attention mask in content and query streams in two-stream self-attention, which correspond to the blue, green and black lines in (a). Since some attention masks in content and query stream are overlapped, we use black lines to denote them in (a). Each row in (b) represents the attention mask for a query position and each column represents a key/value position. The predicted part $x_{z_{>c}} = (x_4, x_6, x_2)$ is predicted by the query stream.

2.3 Our Proposed Method

Figure 2 illustrates the key idea of MPNet. The training objective of MPNet is

$$\mathbb{E}_{z \in \mathcal{Z}_n} \sum_{t=c+1}^{n} \log P(x_{z_t} | x_{z_{<t}}, M_{z_{>c}}; \theta). \quad (5)$$

As can be seen, MPNet conditions on $x_{z_{<t}}$ (the tokens preceding the current predicted token x_{z_t}) rather than only the non-predicted tokens $x_{z_{<=c}}$ in MLM as shown in Equation 3; comparing with PLM as shwon in Equation 4, MPNet takes more information (i.e., the mask symbol $[M]$ in position $z_{>c}$) as inputs. Although the objective seems simple, it is challenging to implement the model efficiently. To this end, we describe several key designs of MPNet in the following paragraphs.

Input Tokens and Positions For a token sequence $x = (x_1, x_2, \cdots, x_6)$ with length $n = 6$, we randomly permute the sequence and get a permuted order $z = (1, 3, 5, 4, 6, 2)$ and a permuted sequence $x_z = (x_1, x_3, x_5, x_4, x_6, x_2)$, where the length of the non-predicted part is $c = 3$, the non-predicted part is $x_{z_{<=c}} = (x_1, x_3, x_5)$, and the predicted part is $x_{z_{>c}} = (x_4, x_6, x_2)$. Additionally, we add mask tokens $M_{z_{>c}}$ right before the predicted part, and obtain the new input tokens $(x_{z_{<=c}}, M_{z_{>c}}, x_{z_{>c}}) = (x_1, x_3, x_5, [M], [M], [M], x_4, x_6, x_2)$ and the corresponding position sequence $(z_{<=c}, z_{>c}, z_{>c}) = (p_1, p_3, p_5, p_4, p_6, p_2, p_4, p_6, p_2)$, as shown in Figure 2a. In MPNet, $(x_{z_{<=c}}, M_{z_{>c}}) = (x_1, x_3, x_5, [M], [M], [M])$ are taken as the non-predicted part, and $x_{z_{>c}} = (x_4, x_6, x_2)$ are taken as the predicted part. For the non-predicted part $(x_{z_{<=c}}, M_{z_{>c}})$, we use bidirectional modeling (Devlin et al., 2019) to extract the representations, which is illustrated as the light grey lines in Figure 2a. Next, we describe how to model the dependency among the predicted part in next paragraph.

Modeling Output Dependency with Two-Stream Self-Attention For the predicted part $x_{z_{>c}}$, since the tokens are in permuted order, the next predicted token could occur in any position, which makes it difficult for normal autoregressive prediction. To this end, we follow PLM to adopt two-stream self-attention (Yang et al., 2019) to autoregressively predict the tokens, which is illustrated in Figure 3. In two-stream self-attention, the query stream can only see the previous tokens and positions as well as current position but cannot see current token, while the content stream can see all the previous and current tokens and positions, as shown in Figure 2a. For more details about two-stream self-attention, please refer to Yang et al. (2019). One drawback of two-stream self-attention in PLM is that it can only see the the previous

Objective	Factorization
MLM (BERT)	$\log P(\text{sentence} \mid \text{the task is [M] [M]}) + \log P(\text{classification} \mid \text{the task is [M] [M]})$
PLM (XLNet)	$\log P(\text{sentence} \mid \text{the task is}) + \log P(\text{classification} \mid \text{the task is sentence})$
MPNet	$\log P(\text{sentence} \mid \text{the task is [M] [M]}) + \log P(\text{classification} \mid \text{the task is sentence [M]})$

Table 1: An example sentence "the task is sentence classification" to illustrate the conditional information of MLM, PLM and MPNet.

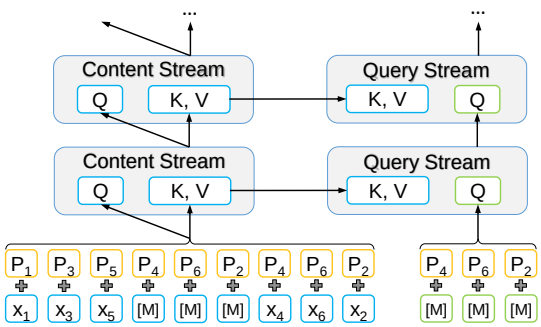


Figure 3: The two-stream self-attention mechanism used in MPNet, where the query stream re-uses the hidden from the content stream to compute key and value.

tokens in the permuted sequence, but does not know the position information of the full sentence during the autoregressive pre-training, which brings discrepancy between pre-training and fine-tuning. To address this limitation, we modify it with position compensation as described next.

Reducing Input Discrepancy with Position Compensation We propose position compensation to ensure the model can see the full sentence, which is more consistent with downstream tasks. As shown in Figure 2b, we carefully design the attention masks for the query and content stream to ensure each step can always see n tokens, where n is the length of original sequence (in the above example, $n = 6$)[4]. For example, when predicting token $x_{z_5} = x_6$, the query stream in the original two-stream attention (Yang et al., 2019) takes mask token $M_{z_5} = [M]$ and position $p_{z_5} = p_6$ as the attention query, and can only see previous tokens $x_{z_{<5}} = (x_1, x_3, x_5, x_4)$ and positions $p_{z_{<5}} = (p_1, p_3, p_5, p_4)$ in the content stream, but cannot see positions $p_{z_{>=5}} = (p_6, p_2)$ and thus miss the full-sentence information. Based on our position compensation, as shown in the second last line of the query stream in Figure 2b, the query stream can see additional tokens $M_{z>=5} = ([M], [M])$ and positions $p_{z_{>=5}} = (p_6, p_2)$. The position compensation in the content stream follows the similar idea as shown in Figure 2b. In this way, we can greatly reduce the input discrepancy between pre-training and fine-tuning.

[4]One trivial solution is to let the model see all the input tokens, i.e., $115\% * n$ tokens, but introduces new discrepancy as the model can only see $100\% * n$ tokens during fine-tuning.

Objective	# Tokens	# Positions
MLM (BERT)	85%	100%
PLM (XLNet)	92.5%	92.5%
MPNet	92.5%	100%

Table 2: The percentage of input information (tokens and positions) leveraged in MLM, PLM and MPNet, assuming they predict the same amount (15%) of tokens.

2.4 Advantage

The main advantage of MPNet over BERT and XLNet is that it conditions on more information while predicting a masked token, which leads to better learnt representations and less discrepancy with downstream tasks.

As shown in Table 1, we take a sentence [The, task, is, sentence, classification] as an example to compare the condition information of MPNet/BERT/XLNet. Suppose we mask two words [sentence, classification] for prediction. As can be seen, while predicting a masked word, MPNet conditions on all the position information to capture a global view of the sentence (e.g., the model knows that there two missing tokens to predict, which is helpful to predict two tokens "sentence classification" instead of three tokens "sentence pair classification"). Note that PLM does not have such kind of information. Furthermore, to predict a word, MPNet conditions on all preceding tokens including the masked and predicted ones for better context modeling (e.g., the model can better predict "classification" given previous token "sentence", instead of predicting "answering" as if to predict "question answering"). In contrast, MLM does not condition on other masked tokens.

Based on the above example, we can derive Ta-

ble 2, which shows how much conditional information is used on average to predict a masked token in each pre-training objective. We assume all the three objectives mask and predict the same amount of tokens (15%), following the common practice in BERT (Devlin et al., 2019) and XLNet (Yang et al., 2019)[5]. As can be seen, MLM conditions on 85% tokens and 100% positions since masked tokens contains position information; PLM conditions on all the 85% unmasked tokens and positions and $15\%/2 = 7.5\%$ masked tokens and positions[6], resulting in 92.5% tokens and positions in total. MPNet conditions on 92.5% tokens similar to PLM, but 100% positions like that in MLM thanks to the position compensation.

To summarize, MPNet utilizes the most information while predicting masked tokens. On the one hand, MPNet can learn better representations with more information as input; on the other hand, MPNet has the minimal discrepancy with downstream language understanding tasks since 100% token and position information of an input sentence is available to a model for those tasks (e.g., sentence classification tasks).

3 Experiments and Results

3.1 Experimental Setup

We conduct experiments under the BERT base setting ($\text{BERT}_{\text{BASE}}$) (Devlin et al., 2019), where the model consists of 12 transformer layers, with 768 hidden size, 12 attention heads as 12, and 110M model parameters in total. For the pre-training objective of MPNet, we randomly permute the sentence following PLM (Yang et al., 2019)[7], choose the rightmost 15% tokens as the predicted tokens, and prepare mask tokens following the same 8:1:1 replacement strategy in BERT (Devlin et al., 2019). Additionally, we also apply whole word mask (Cui et al., 2019) and relative positional embedding (Shaw et al., 2018)[8] in our model pre-training since these tricks have been successfully validated in previous works (Yang et al., 2019; Raffel et al., 2019b).

For pre-training corpus, we follow the data used in RoBERTa (Liu et al., 2019a), which includes: Wikipedia and BooksCorpus (Zhu et al., 2015), OpenWebText (Radford et al., 2019a), CC-News (Liu et al., 2019b) and Stories (Trinh and Le, 2018), with 160GB data size in total. We use a sub-word dictionary with 30K BPE codes in BERT (Devlin et al., 2019) to tokenize the sentences. We limit the length of sentences in each mini-batch up to 512 tokens following the previous practice (Liu et al., 2019b; Joshi et al., 2019) and use a batch size of 8192 sentences. We use Adam (Kingma and Ba, 2014) with $\beta_1 = 0.9$, $\beta_2 = 0.98$ and $\epsilon = 1e-6$. We pre-train our model for 500K steps to be comparable with state-of-the-art models like XLNet (Yang et al., 2019), RoBERTa (Liu et al., 2019a) and ELECTRA (Clark et al., 2020). We use 32 NVIDIA Tesla V100 GPUs, with 32GB memory and FP16 for pre-training, which is estimated to take 35 days[9].

During fine-tuning, we do not use query stream in two-stream self-attention and use the original hiddens to extract context representations following Yang et al. (2019). The fine-tuning experiments on each downstream tasks are conducted 5 times and the median value is chosen as the final result. For experimental comparison, we mainly compare MPNet with previous state-of-the-art pre-trained models using the same $\text{BERT}_{\text{BASE}}$ setting unless otherwise stated. We will also pre-train MPNet in $\text{BERT}_{\text{LARGE}}$ setting in the future.

3.2 Results on GLUE Benchmark

The General Language Understanding Evaluation (GLUE) (Wang et al., 2019) is a collection of 9 natural language understanding tasks, which include two single-sentence tasks (CoLA (Warstadt et al., 2018), SST-2 (Socher et al., 2013)), three similarity and paraphrase tasks (MRPC (Dolan and Brockett, 2005), STS-B (Cer et al., 2017), QQP), four inference tasks (MNLI (Williams et al., 2018), QNLI (Rajpurkar et al., 2016), RTE (Dagan et al., 2006), WNLI (Levesque et al., 2012)). We follow RoBERTa hyper-parameters for single-task fine-tuning, where RTE, STS-B and MRPC are started from the MNLI fine-tuned model.

We list the results of MPNet and other existing strong baselines in Table 3. All of the listed results are reported in $\text{BERT}_{\text{BASE}}$ setting and from

[5] XLNet masks and predicts 1/7 tokens, which are close to 15% predicted tokens.

[6] PLM is actually a language model, which predicts the i-th token given the previous $i-1$ tokens. Therefore, the number of tokens can be leveraged on average is $(n-1)/2$ where n is number of predicted tokens.

[7] Note that we only improve upon PLM in XLNet, and we do not use long-term memory in XLNet.

[8] We follow Raffel et al. (2019b) to adopt a shared relative position embedding across each layer for efficiency.

[9] Note that MPNet is still under pre-training with only 270K steps, but has already outperformed previous models. We will update the results once pre-trained for 500K steps.

	MNLI	QNLI	QQP	RTE	SST	MRPC	CoLA	STS	Avg
Single model on dev set									
BERT (Devlin et al., 2019)	84.5	91.7	91.3	68.6	93.2	87.3	58.9	89.5	83.1
XLNet (Yang et al., 2019)	86.8	91.7	91.4	74.0	94.7	88.2	60.2	89.5	84.5
RoBERTa (Liu et al., 2019a)	87.6	92.8	**91.9**	78.7	94.8	90.2	63.6	**91.2**	86.4
MPNet	**88.5**	**93.3**	**91.9**	**85.2**	**95.4**	**91.5**	**65.0**	90.9	**87.7**
Single model on test set									
BERT (Devlin et al., 2019)	84.6	90.5	89.2	66.4	93.5	84.8	52.1	87.1	79.9
ELECTRA (Clark et al., 2020)	**88.5**	**93.1**	89.5	75.2	**96.0**	88.1	**64.6**	**91.0**	85.8
MPNet	**88.5**	93.0	**89.6**	**80.5**	95.6	**88.2**	64.0	90.7	**86.3**

Table 3: Comparisons between MPNet and the previous strong pre-trained models under $\text{BERT}_{\text{BASE}}$ setting on the dev and test set of GLUE tasks. We only list the results on each set that are available in the published papers. STS is reported by Pearman correlation, CoLA is reported by Matthew's correlation, and other tasks are reported by accuracy.

SQuAD v1.1	**EM**	**F1**
BERT (Devlin et al., 2019)	80.8	88.5
RoBERTa (Liu et al., 2019a)	84.6	91.5
MPNet	**86.8**	**92.5**
SQuAD v2.0	**EM**	**F1**
BERT (Devlin et al., 2019)	73.7	76.3
XLNet (Yang et al., 2019)	80.2	-
RoBERTa (Liu et al., 2019a)	80.5	83.7
MPNet	**82.8**	**85.6**

Table 4: Comparison between MPNet and the previous strong pre-trained models under $\text{BERT}_{\text{BASE}}$ setting on the SQuAD dev set. We report results by exact match (EM) and F1 score.

single model without any data augmentation for fair comparisons. On the dev set of GLUE tasks, MPNet outperforms BERT (Devlin et al., 2019), XLNet (Yang et al., 2019) and RoBERTa (Liu et al., 2019a) by 4.6, 3.2, 1.3 points on average. On the test set of GLEU tasks, MPNet outperforms ELECTRA (Clark et al., 2020), which achieved previous state-of-the-art accuracy on a variety of language understanding tasks, by 0.5 point on average, demonstrating the advantages of MPNet for language understanding. Since MPNet is still under pre-training, we believe there still exists large improvement space for MPNet.

3.3 Results on Question Answering (SQuAD)

The Stanford Question Answering Dataset (SQuAD) task requires to extract the answer span from the provided context based on the question. We evaluate our model on SQuAD v1.1 (Rajpurkar et al., 2016) and SQuAD v2.0 (Rajpurkar et al., 2018). SQuAD v1.1 always exists the corresponding answer for each question and the corresponding context, while some questions do not have the corresponding answer in SQuAD v2.0. For v1.1, we add a classification layer on the outputs from the pre-trained model to predict whether the token is a start or end position of the answer. For v2.0, we additionally add a binary classification layer to predict whether the answer exists.

The results of MPNet on SQuAD dev set are reported on Table 4. All of the listed results are reported in $\text{BERT}_{\text{BASE}}$ setting and from single model without any data augmentation for fair comparisons. It can be seen that MPNet outperforms BERT, XLNet and RoBERTa by a large margin, both in SQuAD v1.1 and v2.0, which are consistent with the results on GLUE tasks, demonstrating the advantages of MPNet.

3.4 Results on RACE

The ReAding Comprehension from Examinations (RACE) (Lai et al., 2017) is a large-scale dataset collected from the English examinations from middle and high school students. In RACE, each passage has multiple questions and each question has

[10] http://www.qizhexie.com/data/RACE_leaderboard.html

Model Setting	SQuAD v1.1		SQuAD v2.0		GLUE	
	EM	F1	EM	F1	MNLI	SST-2
MPNet	**85.0**	**91.4**	**80.5**	**83.3**	**86.2**	**94.0**
– position compensation (= PLM)	83.0	89.9	78.5	81.0	85.6	93.4
– permutation (= MLM + output dependency)	84.1	90.6	79.2	81.8	85.7	93.5
– permutation & output dependency (= MLM)	82.0	89.5	76.8	79.8	85.6	93.3

Table 5: Ablation study of MPNet under $\text{BERT}_{\text{BASE}}$ setting on the dev set of SQuAD tasks (v1.1 and v2.0) and GLUE tasks (MNLI and SST-2). The experiments in ablation study are all pre-trained on the Wikipedia and BooksCorpus (16GB size) for 1M steps, with a batch size of 256 sentences, each sentence with up to 512 tokens.

	RACE			IMDB
	Acc.	Middle	High	Err.
BERT*	65.0	71.7	62.3	5.4
XLNet*	66.8	-	-	4.9
MPNet*	70.4	**76.8**	67.7	4.8
MPNet	**72.0**	76.3	**70.3**	**4.4**

Table 6: Results on the RACE and IMDB test set under $\text{BERT}_{\text{BASE}}$ setting. For RACE, the results of BERT are from the RACE leaderboard[10]and the results of XLNet are obtained from the original paper (Yang et al., 2019). "Middle" and "High" denote the accuracy on the middle school set and high school set in RACE. For IMDB, the result of BERT is from Sun et al. (2019) and the result of XLNet is ran by ourselves with only PLM pre-training objective but no long context memory (Yang et al., 2019). "*" represents pre-training only on Wikipedia and BooksCorpus (16GB size).

four choices. The task is to select the correct choice based on the given options.

The results on RACE task are listed in Table 6. We can only find the results from BERT and XLNet pre-trained on Wikipedia and BooksCorpus (16GB data). For a fair comparison, we also pre-train MPNet on 16GB data (marked as * in Table 6). MPNet greatly outperforms BERT and XLNet across the three metrics, demonstrating the advantages of MPNet. When pre-training MPNet with the default 160GB data, an additional 1.6 points gain (72.0 vs. 70.4) can be further achieved.

3.5 Results on IMDB

We further study MPNet on the IMDB text classification task (Maas et al., 2011), which contains over 50,000 movie reviews for binary sentiment classification. The results are reported in Table 6. It can be seen that MPNet trained on Wikipedia and BooksCorpus (16GB data) outperform BERT and PLM (XLNet) by 0.6 and 0.1 point. When pre-training with 160GB data, MPNet achieves additional 0.4 point gain.

3.6 Ablation Study

We further conduct ablation studies to analyze several design choices in MPNet, including introducing dependency among predicted tokens to MLM, introducing position compensation to PLM, etc. The results are shown in Table 5. We have several observations:

- After removing position compensation, MPNet degenerates to PLM, and its accuracy drops by 0.6-2.3 points in downstream tasks. This demonstrates the effectiveness of position compensation and the advantage of MPNet over PLM.
- After removing permutation operation but still keeping the dependency among the predicted tokens with two-stream attention (*i.e.*, MLM + output dependency), the accuracy drops slightly by 0.5-1.7 points. This verifies the gain of permutation used in MPNet.
- If removing both permutation and output dependency, MPNet degenerates to MLM, and its accuracy drops by 0.5-3.7 points, demonstrating the advantage of MPNet over MLM.

4 Conclusion

In this paper, we proposed MPNet, a novel pre-training method that addresses the problems of MLM in BERT and PLM in XLNet. MPNet leverages the dependency among the predicted tokens through permuted language modeling and makes the model to see auxiliary position information to reduce the discrepancy between pre-training and fine-tuning. Experiments on various tasks demonstrate that MPNet outperforms MLM and PLM, as well as previous strong pre-trained models such as

BERT, XLNet, RoBERTa by a large margin. In the future, we will pre-train MPNet with larger model setting for better performance, and apply MPNet on more diverse language understanding tasks.

References

Daniel Cer, Mona Diab, Eneko Agirre, Iñigo Lopez-Gazpio, and Lucia Specia. 2017. SemEval-2017 task 1: Semantic textual similarity multilingual and crosslingual focused evaluation. In *Proceedings of the 11th International Workshop on Semantic Evaluation (SemEval-2017)*, pages 1–14.

Kevin Clark, Minh-Thang Luong, Quoc V Le, and Christopher D Manning. 2020. Electra: Pre-training text encoders as discriminators rather than generators. *arXiv preprint arXiv:2003.10555*.

Yiming Cui, Wanxiang Che, Ting Liu, Bing Qin, Ziqing Yang, Shijin Wang, and Guoping Hu. 2019. Pre-training with whole word masking for chinese BERT. *CoRR*, abs/1906.08101.

Ido Dagan, Oren Glickman, and Bernardo Magnini. 2006. The pascal recognising textual entailment challenge. In *Machine Learning Challenges. Evaluating Predictive Uncertainty, Visual Object Classification, and Recognising Tectual Entailment*, pages 177–190.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *NAACL*, pages 4171–4186.

William B. Dolan and Chris Brockett. 2005. Automatically constructing a corpus of sentential paraphrases. In *Proceedings of the Third International Workshop on Paraphrasing (IWP2005)*.

Li Dong, Nan Yang, Wenhui Wang, Furu Wei, Xiaodong Liu, Yu Wang, Jianfeng Gao, Ming Zhou, and Hsiao-Wuen Hon. 2019. Unified language model pre-training for natural language understanding and generation. In *NIPS*, pages 13042–13054.

Mandar Joshi, Danqi Chen, Yinhan Liu, Daniel S Weld, Luke Zettlemoyer, and Omer Levy. 2019. Spanbert: Improving pre-training by representing and predicting spans. *arXiv preprint arXiv:1907.10529*.

Diederik P. Kingma and Jimmy Ba. 2014. Adam: A method for stochastic optimization.

Guokun Lai, Qizhe Xie, Hanxiao Liu, Yiming Yang, and Eduard Hovy. 2017. RACE: Large-scale ReAding comprehension dataset from examinations. In *EMNLP*, pages 785–794.

Hector J. Levesque, Ernest Davis, and Leora Morgenstern. 2012. The Winograd Schema Challenge. In *Proceedings of the Thirteenth International Conference on Principles of Knowledge Representation and Reasoning*, pages 552–561. Rome, Italy.

Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019a. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*.

Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019b. Roberta: A robustly optimized BERT pretraining approach. *CoRR*, abs/1907.11692.

Andrew L. Maas, Raymond E. Daly, Peter T. Pham, Dan Huang, Andrew Y. Ng, and Christopher Potts. 2011. Learning word vectors for sentiment analysis. In *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*, pages 142–150.

Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. 2018. Improving language understanding by generative pre-training. *URL https://s3-us-west-2. amazonaws. com/openai-assets/researchcovers/languageunsupervised/language understanding paper. pdf*.

Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019a. Language models are unsupervised multitask learners.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019b. Language models are unsupervised multitask learners. *OpenAI Blog*, 1(8).

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. 2019a. Exploring the limits of transfer learning with a unified text-to-text transformer. *arXiv preprint arXiv:1910.10683*.

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2019b. Exploring the limits of transfer learning with a unified text-to-text transformer. *arXiv e-prints*.

Pranav Rajpurkar, Robin Jia, and Percy Liang. 2018. Know what you don't know: Unanswerable questions for SQuAD. In *ACL*, pages 784–789.

Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. SQuAD: 100,000+ questions for machine comprehension of text. In *EMNLP*, pages 2383–2392.

Peter Shaw, Jakob Uszkoreit, and Ashish Vaswani. 2018. Self-attention with relative position representations. In *NAACL*, pages 464–468.

Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D. Manning, Andrew Ng, and Christopher Potts. 2013. Recursive deep models for semantic compositionality over a sentiment treebank. In *EMNLP*, pages 1631–1642.

Kaitao Song, Xu Tan, Tao Qin, Jianfeng Lu, and Tie-Yan Liu. 2019. MASS: Masked sequence to sequence pre-training for language generation. In *ICML*, pages 5926–5936.

Chi Sun, Xipeng Qiu, Yige Xu, and Xuanjing Huang. 2019. How to fine-tune BERT for text classification? *CoRR*, abs/1905.05583.

Trieu H. Trinh and Quoc V. Le. 2018. A simple method for commonsense reasoning. *CoRR*, abs/1806.02847.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Ł ukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *NIPS*, pages 5998–6008.

Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. 2019. GLUE: A multi-task benchmark and analysis platform for natural language understanding. In *ICLR*.

Alex Warstadt, Amanpreet Singh, and Samuel R. Bowman. 2018. Neural network acceptability judgments. *CoRR*, abs/1805.12471.

Adina Williams, Nikita Nangia, and Samuel Bowman. 2018. A broad-coverage challenge corpus for sentence understanding through inference. In *NAACL*, pages 1112–1122.

Zhilin Yang, Zihang Dai, Yiming Yang, Jaime Carbonell, Russ R Salakhutdinov, and Quoc V Le. 2019. Xlnet: Generalized autoregressive pretraining for language understanding. In *NIPS*, pages 5754–5764.

Yukun Zhu, Ryan Kiros, Rich Zemel, Ruslan Salakhutdinov, Raquel Urtasun, Antonio Torralba, and Sanja Fidler. 2015. Aligning books and movies: Towards story-like visual explanations by watching movies and reading books. In *ICCV*, pages 19–27.

A Pre-training Hyper-parameters

The pre-training hyper-parameters are reported in Table 7.

Hyper-parameter	Value
Number of Layers	12
Hidden Size	768
Filter Size	3072
Attention heads	12
Dropout	0.1
Weight Decay	0.01

Table 7: Pre-training hyper-parameters for $\text{BERT}_{\text{BASE}}$ setting.

B Fine-tuning Hyper-parameters

The fine-tuning hyper-parameters are reported in Table 8.

C More Ablation Studies

We further conduct more experiments to analyze the effect of whole word mask and relative positional embedding. The results are reported in Table 9.

Hyper-parameter	RACE	SQuAD	GLUE
Learning Rate	1.5e-5	2e-5	1e-5,2e-5,3e-5
Batch Size	16	48	16, 32
Weight Decay	0.1	0.01	0.1
Epochs	5	4	10, 15
Learning Rate Decay	Linear	Linear	Linear
Warmup Ratio	0.06	0.06	0.06

Table 8: Fine-tuning hyper-parameters for RACE, SQuAD and GLUE.

	SQuAD v1.1		**SQuAD v2.0**		**GLUE**	
Model Setting	EM	F1	EM	F1	MNLI	SST-2
MPNet	**85.0**	**91.4**	**80.5**	**83.3**	**86.2**	**94.0**
− whole word mask	84.0	90.5	79.9	82.5	85.6	93.8
− relative positional embedding	84.0	90.3	79.5	82.2	85.3	93.6

Table 9: Ablation study of MPNet under $\text{BERT}_{\text{BASE}}$ setting on the dev set of SQuAD tasks (v1.1 and v2.0) and GLUE tasks (MNLI and SST-2).